

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
from google.colab import files
uploaded = files.upload()
```

[Choose Files](#) heart.csv

heart.csv(text/csv) - 38114 bytes, last modified: 2/19/2026 - 100% done
Saving heart.csv to heart.csv

```
import pandas as pd

df = pd.read_csv("heart.csv")
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target	
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0	
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0	
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0	
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0	
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.info()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    age         1025 non-null   int64
1    sex         1025 non-null   int64
2    cp          1025 non-null   int64
3    trestbps    1025 non-null   int64
4    chol        1025 non-null   int64
5    fbs         1025 non-null   int64
6    restecg     1025 non-null   int64
7    thalach     1025 non-null   int64
8    exang       1025 non-null   int64
9    oldpeak     1025 non-null   float64
10   slope       1025 non-null   int64
11   ca          1025 non-null   int64
12   thal        1025 non-null   int64
13   target      1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

```

      0
age    0
sex    0
cp     0
trestbps 0
chol   0
fbs    0
restecg 0
thalach 0
exang   0
oldpeak 0
slope   0
ca      0
thal    0
target  0

```

dtype: int64

```
X = df.drop("target", axis=1)
y = df["target"]
```

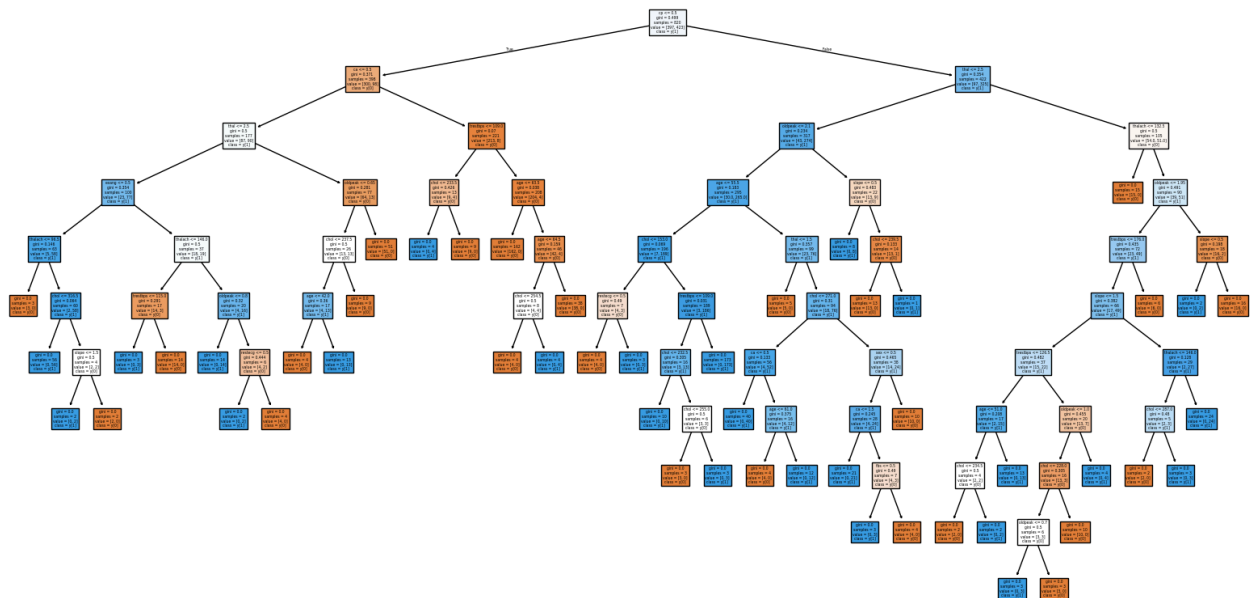
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
```

Decision Tree Accuracy: 0.9853658536585366

```
plt.figure(figsize=(20,10))
plot_tree(dt, feature_names=X.columns, class_names=True, filled=True)
plt.show()
```



```
dt_limited = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_limited.fit(X_train, y_train)

y_pred_limited = dt_limited.predict(X_test)
print("Limited Depth Accuracy:", accuracy_score(y_test, y_pred_limited))
```

Limited Depth Accuracy: 0.7804878048780488

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

Random Forest Accuracy: 0.9853658536585366

```
importances = rf.feature_importances_
feature_names = X.columns

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

print(importance_df)
```

	Feature	Importance
2	cp	0.135072
11	ca	0.127327
7	thalach	0.122169
9	oldpeak	0.121905
12	thal	0.110518
0	age	0.077908
4	chol	0.074822
3	trestbps	0.071171
8	exang	0.057594
10	slope	0.045782
1	sex	0.028731
6	restecg	0.018557
5	fbs	0.008444

```
plt.figure(figsize=(10,6))
sns.barplot(x="Importance", y="Feature", data=importance_df)
plt.title("Feature Importance - Random Forest")
```

```
plt.show()
```

